

ENUNCIADO

Crear un programa que a través de hilos realice las siguientes tareas

a) Estructura de datos

Crea una matriz de enteros con tamaño ROWSxCOLS, con valores iniciales ROWS = 3 y COLS = 3. Crea también una variable global entera llamada `buffer_temporal`.

b) Creación de hilos

Crea 3 hilos que se llamarán "Coordinador", "Lector" y "Filtro". Estos hilos deben permanecer vivos hasta el final del programa.

c) Dinámica de Cadena de Montaje (A -> B -> C)

El Coordinador gestiona el avance por las posiciones de la matriz de forma secuencial (recorriendo filas y columnas). Por cada celda, iniciará un ciclo de trabajo en cadena.

d) Tareas de los hilos

El Coordinador avisa al Lector para que empiece a trabajar. El Lector pide un número por teclado al usuario, lo guarda en `buffer_temporal` y avisa al hilo Filtro. (El Lector no avisa al Coordinador). El Filtro evalúa el `buffer_temporal`: Si el número es par, guarda un 1 en la posición actual de la matriz. Si el número es impar, guarda un 0 en la posición actual de la matriz. Una vez guardado, el Filtro avisa al Coordinador de que la celda está lista.

e) Sincronización

Estrictamente prohibido emplear flags, semáforos o mutex. Únicamente señales: SIGUSR1, SIGUSR2 y SIGALRM.

f) Mostrar resultado

Cuando la matriz está llena por completo, el Coordinador muestra el contenido de la matriz en formato tabular.

g) Finalización ordenada

Una vez mostrada la matriz, el Coordinador elimina primero a Filtro, luego a Lector y por último a sí mismo. h) Terminación del programa El proceso principal (main) esperará a que el programa termine sus tareas.

ENUNCIADO

Desarrolla un programa en C que simule un sistema donde un proceso Padre recolecta datos y un proceso Hijo usa hilos para procesarlos, implementando la siguiente funcionalidad:

a) Creación de comunicación y procesos (**pipes** y **fork**)

El proceso original debe crear una tubería (**pipe**) unidireccional. Inmediatamente después, debe crear un proceso hijo usando **fork()**.

b) La tarea del Padre (Escritura y **wait** con **WIFEXITED**)

El proceso Padre tiene un array con 5 números enteros predefinidos: {10, 20, 30, 40, 50}.

- Debe enviar este array completo al proceso Hijo a través del **pipe** y cerrar los descriptores correspondientes.
- Luego, el Padre debe esperar a que el Hijo termine usando **wait()**.
- Una vez el Hijo termine, el Padre debe comprobar usando **WIFEXITED()** si la terminación fue normal, e imprimir por pantalla el **PID** del hijo recogido y el código numérico que devolvió.

c) La tarea del Hijo (Lectura e **hilos**)

El proceso Hijo debe leer los 5 enteros provenientes del **pipe** y guardarlos en un array global.

- Una vez leídos, el Hijo creará **3 hilos** (operarios).
- El proceso Hijo (el hilo principal) debe esperar a que los 3 hilos terminen su trabajo usando **pthread_join()**.
- Al finalizar, el proceso Hijo imprimirá el resultado total de la suma (ver punto d) y terminará enviando mediante **exit()** el código 77.

d) Sincronización de Tareas (**mutex** y **semáforos**)

Los 3 hilos operarios tienen la misión de ir cogiendo números del array global y sumarlos en una variable global llamada **suma_total**.

- **Semáforo:** Los hilos no pueden empezar a procesar ni sumar nada hasta que el Hijo haya terminado de leer los datos del **pipe**. Utiliza un **semáforo** para bloquear a los hilos al inicio de su función, y que el Hijo los libere (haciendo señales al semáforo) cuando los datos estén listos en el array.
- **Mutex:** Como varios hilos van a leer el array y a modificar la variable **suma_total** a la vez, debes usar un **mutex** para evitar una *condición de carrera*. El mutex debe proteger la extracción del número y la suma.